Fleet Finance Management System
Review 3

DA 3 / BCSE406L / NoSQL Databases / D2+TD2 / Fall 26-27 / Dr Geetha S

Abhinavanagarajan A / 23BCE1123 / Redis Implementation
Rakshana V / 23BCE1620 / MongoDB Implementation
Vishnu Swaroop G / 23BCE1129 / Integration to DBs

[1] Project overview

Problem: Small and mid-sized private intercity bus operators record each bus's daily trip ("single") collections, odometer readings and diesel purchases on paper ledgers. This gives them no real-time view of revenue trends and delays the discovery of weak routes, trips or buses. Distance, fuel and fare data are never correlated. Flat averages also misread the normal Sunday spike or weekday dip as an anomaly.

Solution: a web application on a hybrid NoSQL architecture:

- **MongoDB** is the durable source of truth. It stores one document per bus per day, with the singles, odometer, fuel fill, computed summary and performance evaluation embedded.
- **Redis** is an in-memory layer holding live fleet totals, rolling averages and a revenue leaderboard.
- **Flask** is the application layer that connects the two.

Objectives and how they are met

#	Objective	Implementation
1	Digital intake of singles, odometer, fuel	Submit Shift form with any number of singles per day
2	Rolling averages and above/below flags on submission	`bus:rolling:{bus_id}` refreshed on every write; `collection_status` stored in the document
3	Mileage (km/L) vs. the bus's baseline	`mileage_kmpl = total_km / litres`, scored GOOD / AVERAGE / LOW
4	Day-type patterns (Sunday/holiday vs. weekday)	Per-bus `day_type_factors;` Sundays are judged against a Sunday baseline (Q4)
5	Fast fleet ranking, expense and route-margin queries	Redis sorted set leaderboard; Mongo aggregation pipelines

Why NoSQL? A bus-day is a hierarchical record with a variable number of singles. In a relational design it would need joins across DailyEntries, Singles and FuelLogs on every dashboard refresh. A document model stores the whole day as one atomic document, and the number of singles varies per day with no schema migration. Redis then serves the hot read paths from memory.

[2] NoSQL database and data model

Stores: MongoDB database `fleet_finance_db` and Redis db 0.

MongoDB collections

`daily_bus_logs` holds one document per bus per date

```
{
  "_id": "LOG_TN09BC1001_20260824",
  "bus_id": "BUS_101", "registration_no": "TN-09-BC-1001",
  "route_id": "RT_CHN_VLR", "date": "2026-08-24", "day_type":
"WEEKDAY",
  "odometer": {"start_km": 148367, "end_km": 148728, "total_km":
361},
  "singles": [
    {"single_no": 1, "trip_code": "S1_UP", "departure_time": "06:00",
     "collection_inr": …, "passenger_count": …},  … 4 singles in
total
  ],
  "fuel_fill": {"litres": …, "fuel_rate_per_litre": …,
"total_fuel_cost": …, "bunk_name": "HPCL_Walajapet"},
  "other_expense_inr": 0.0,
  "computed_summary": {"gross_collection_inr": 20520.0,
"total_expense_inr": 7480.95,
                       "net_margin_inr": 13039.05, "mileage_kmpl":
4.54},
  "performance_evaluation": {"baseline_avg_collection": 17800.0,
"collection_status": "ABOVE_AVERAGE",
                             "mileage_baseline_kmpl": 4.6,
"mileage_status": "AVERAGE"}
}
```

`buses_master` holds one document per physical bus (5 documents):

```
{"bus_id": "BUS_101", "registration_no": "TN-09-BC-1001", "chassis":
"CH-101-2019",
```

"permit_type": "STAGE_CARRIAGE", "seating_capacity": 52, "route_id": "RT_CHN_VLR",
"route_name": "Chennai-Vellore", "baseline_target_kmpl": 4.6, "baseline_avg_collection": 17800.0,
"day_type_factors": {"WEEKDAY": 1.0, "SUNDAY": 1.4, "HOLIDAY": 1.3}, "default_bunk": "HPCL_Walajapet"}

Embedding vs. referencing

- **Embedded:** singles, odometer, fuel fill and computed summary. They are bounded (3–5 singles per day) and always read together with the day, so one read returns the whole day.
- **Referenced:** `buses_master` is linked by `bus_id`. Static data (chassis, permit, seats, baselines) is shared by 150+ growing logs. Embedding it would copy it thousands of times and make baseline changes hard.

Redis structures

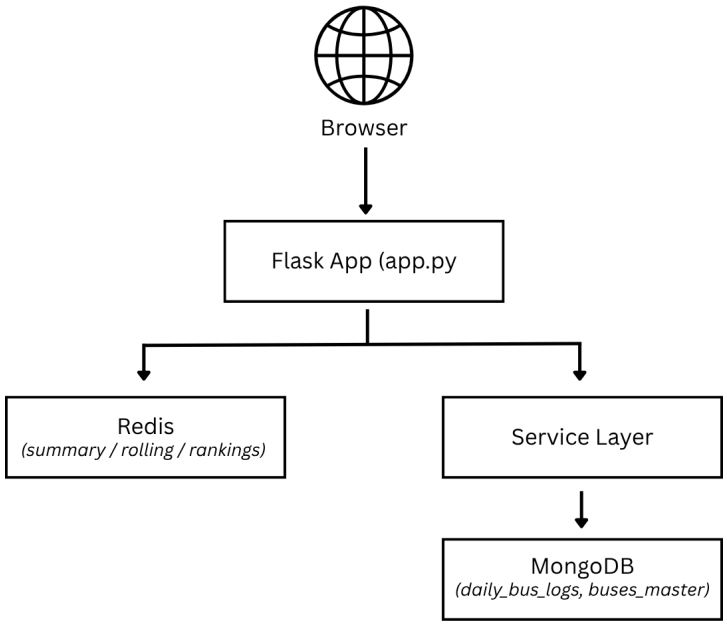
Key	Type	Content	Main commands
`fleet:summary:{date}`	Hash	`gross_collection_inr`, `net_margin_inr`, `buses_reported`	HINCRBYFLOAT, HINCRBY, HGETALL
`bus:rolling:{bus_id}`	Hash	7-day / 30-day / Sunday / weekday averages, 30-day mileage, last log date	HSET, HGETALL
`fleet:rankings:{date}`	Sorted set	member = bus, score = gross	ZADD, ZREVRANGE

Consistency principle. MongoDB is the source of truth and Redis is a derived cache. Every Redis write is a **delta** (new − old), so re-submitting or editing a day never double-counts. `rebuild_all()` can recreate the whole cache from MongoDB, and `audit()` verifies the two stores agree.

[3] Implementation

Technology: Python 3, Flask, PyMongo, redis-py, MongoDB 7, Redis 7

Architecture



Modules

File	Responsibility
`db.py`	Connections, the 5 index definitions, health probes, friendly startup errors
`logic.py`	Business rules: day-type detection, mileage, margin, baseline evaluation
`generate_data.py`	Deterministic seed (random seed 2026): 5 buses, 150 logs, indexes, Redis keys
`service.py`	One code path for “write to MongoDB, then update Redis” (create, update, delete, reads)
`redis_layer.py`	Redis structures, delta updates, rolling refresh, rebuild, audit
`queries_demo.py`	The 12 queries as data specs; the displayed code is generated from the executed spec

`indexing_demo.py`	Index list, `explain()` comparison, timing, scale benchmark
`crud_demo.py`, `run_all.py`	CRUD demonstration and the reproducible end-to-end run with PASS/FAIL checks
`app.py + templates`	Dashboard, Submit Shift, Logs (CRUD), Query Console, Index Lab, Redis Explorer, Health

Shift-submission flow (Application → Database → Query → Result)

1. The form is validated (dates, odometer end > start, positive fuel, at least one single).
2. `logic.build_log` computes gross, expense, net, mileage, day type and the performance evaluation.
3. MongoDB: `replace_one({_id}, doc, upsert=True)`.
4. Redis, in one MULTI/EXEC: `HINCRBYFLOAT` ×2, `HINCRBY`, `ZADD`, then the rolling averages are recomputed from MongoDB.
5. The UI shows a trace of every command executed.

Operation	MongoDB	Redis effect
Create	`replace_one(..., upsert=True)` → inserted	summary, leaderboard and rolling stats created
Read	`find_one` by `_id`; `find` Sundays of BUS_101	–
Update	`update_one` with `$push` (late single) and `$set` (recomputed totals): gross 12,350 → 16,250, singles 3 → 4 (verify)	`HINCRBYFLOAT +3900`, `ZADD`, rolling refreshed
Delete	`delete_one` → `deleted_count: 1`, `find_one` → None	`HINCRBYFLOAT -gross`, `ZREM`

Reliability features

- **Graceful degradation:** if Redis is down, the dashboard falls back to MongoDB queries and says so on screen.
- **Self-healing cache:** one click rebuilds Redis from MongoDB.
- **Optimistic concurrency:** updates apply only if the gross is unchanged since it was read, with up to 3 retries.

- **Verification:** 12 automated tests cover page rendering, all 12 queries, idempotent upsert, update, delete, validation, Redis-down fallback and rebuild.

[4] Important queries

Q1 – Point lookup (compound index `ix_bus_date`)

```
db.daily_bus_logs.findOne({bus_id:"BUS_101", date:"2026-08-24"})
```

Q2 – Anomaly detection (range filter + nested field)

```
db.daily_bus_logs.find({bus_id:"BUS_104",
date:{$gte:"2026-08-01",$lte:"2026-08-31"},

"performance_evaluation.collection_status":"BELOW_AVERAGE"}).sort({da
te:1})
```

Q4 – Day-type aggregation

```
db.daily_bus_logs.aggregate([
 {$group:{_id:{bus_id:"$bus_id",day_type:"$day_type"},
          avg_gross:{$avg:"$computed_summary.gross_collection_inr"},
days_counted:{$sum:1}}},
 {$sort:{"_id.bus_id":1,"_id.day_type":1}}])
```

Q5 – Peak-revenue single per route ($unwind on the embedded array)

```
db.daily_bus_logs.aggregate([{$unwind:"$singles"},
 {$group:{_id:{route_id:"$route_id",trip_code:"$singles.trip_code"},
          avg_single_collection:{$avg:"$singles.collection_inr"},
total_passengers:{$sum:"$singles.passenger_count"}}},
 {$sort:{avg_single_collection:-1}},{$limit:5}])
```

Q7 – Route margin intelligence

```
db.daily_bus_logs.aggregate([{$group:{_id:"$route_id",
  total_net_margin:{$sum:"$computed_summary.net_margin_inr"},
  avg_mileage:{$avg:"$computed_summary.mileage_kmpl"},
trip_days:{$sum:1}}},
 {$sort:{total_net_margin:-1}}])
```

Q9 / Q10 – Redis leaderboard and cached stats

```
ZREVRANGE fleet:rankings:2026-08-24 0 4 WITHSCORES
HGETALL bus:rolling:BUS_101        HGETALL fleet:summary:2026-08-24
```

All 12 queries

#	Purpose	Feature
Q1	Point lookup by bus and date	compound index
Q2	Underperforming days in a date range	nested-field index, range, sort
Q3	Fleet-wide low-mileage flags	nested filter, limit
Q4	Sunday vs. weekday averages	`$group, $avg`
Q5	Peak-revenue single per route	`$unwind, $group`
Q6	Top-3 most profitable bus-days	`sort on a nested key`
Q7	Route margin ranking	`$group, $sum`
Q8	Log → master reference lookup	referencing
Q9	Live leaderboard	Redis sorted set
Q10	Rolling stats and fleet summary	Redis hashes
Q11	Day's logs joined with master data	`$lookup, $unwind, $project`
Q12	Revenue and fuel cost per km	`$group, $divide`

[5] Indexing / Aggregation / NoSQL features used

Indexes

Index	Key	Used by
`ix_bus_date`	`{bus_id:1, date:-1} (compound)`	Q1, rolling-average refresh, dashboard history
`ix_route`	`{route_id:1}`	Q5, Q7

ix_daytype	{day_type:1}	Q4 and Sunday-average refresh
ix_perf_status	{performance_evaluation.collection_status:1} (nested)	Q2, anomaly alerts
ix_bus_master_pk	{bus_id:1}, unique	Q8, one master record per bus

Evidence (Index Lab). explain("executionStats") is run for each query twice: once with the planner's choice and once with a forced collection scan (hint {$natural:1}).

- Planner's choice: IXSCAN, with docs examined equal to docs returned.
- Forced scan: COLLSCAN, with all 150 documents examined.
- Non-indexed field: fuel_fill.bunk_name falls back to COLLSCAN. [Insert docs examined and times from your run.]

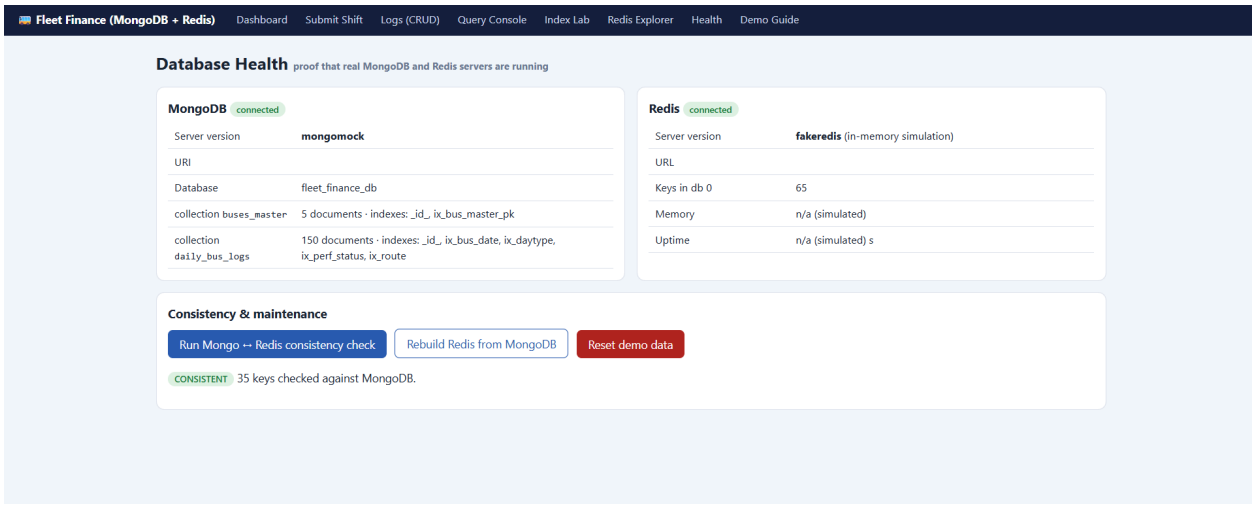
Aggregation

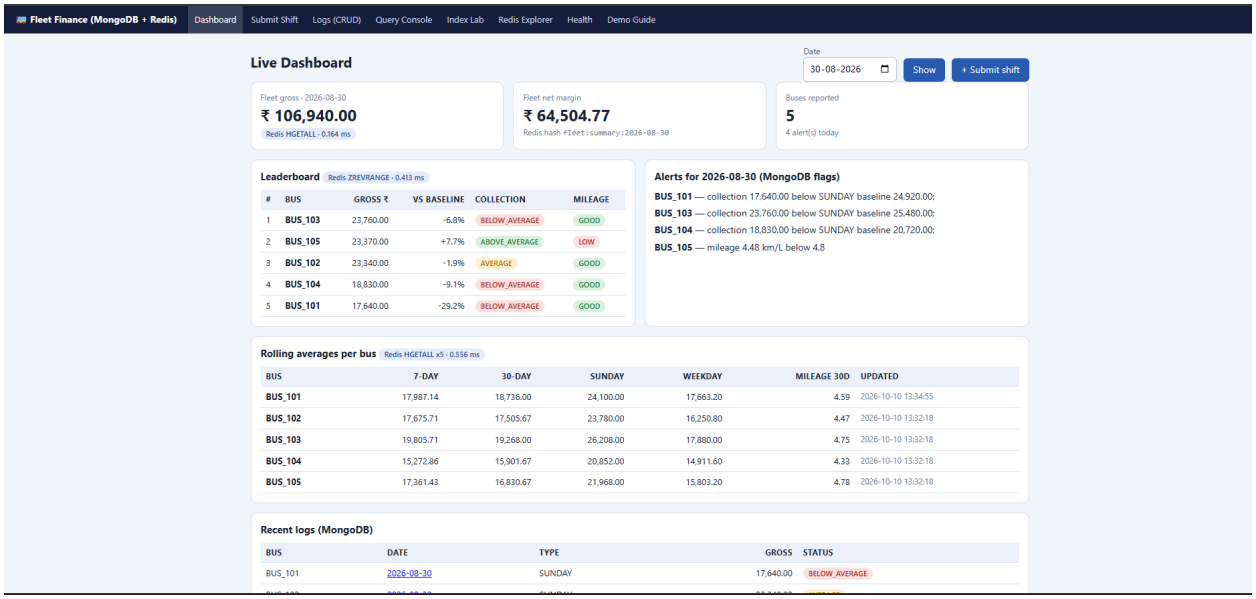
Q4, Q5, Q7, Q11 and Q12 use these pipeline stages: $group, $unwind, $lookup, $project, $sort, $limit, with accumulators $sum, $avg and $divide.

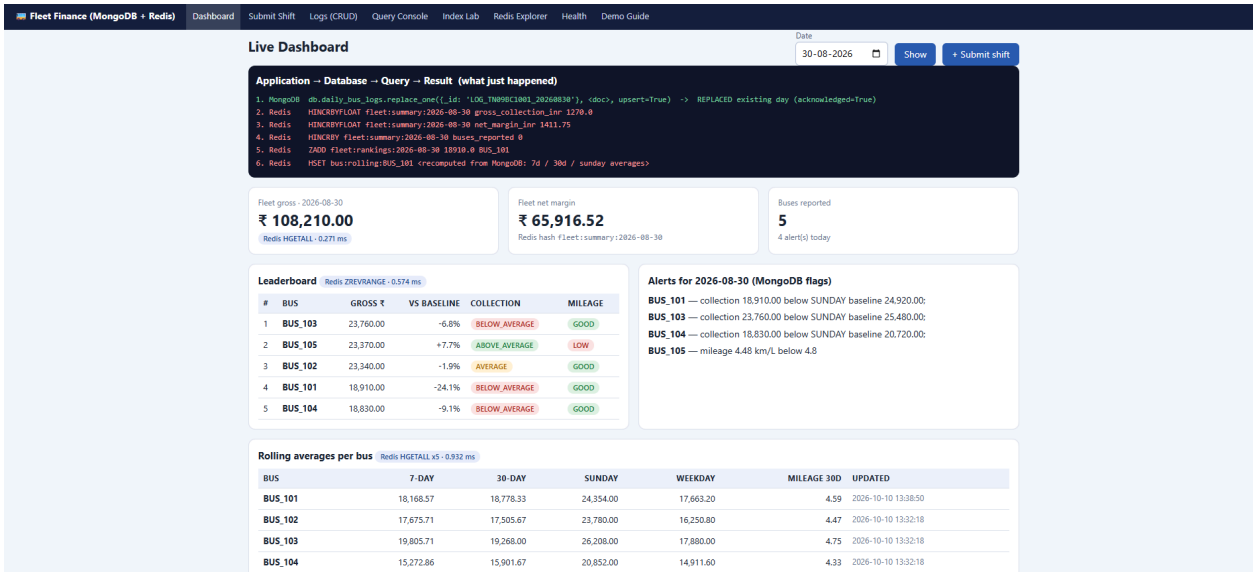
Other NoSQL features

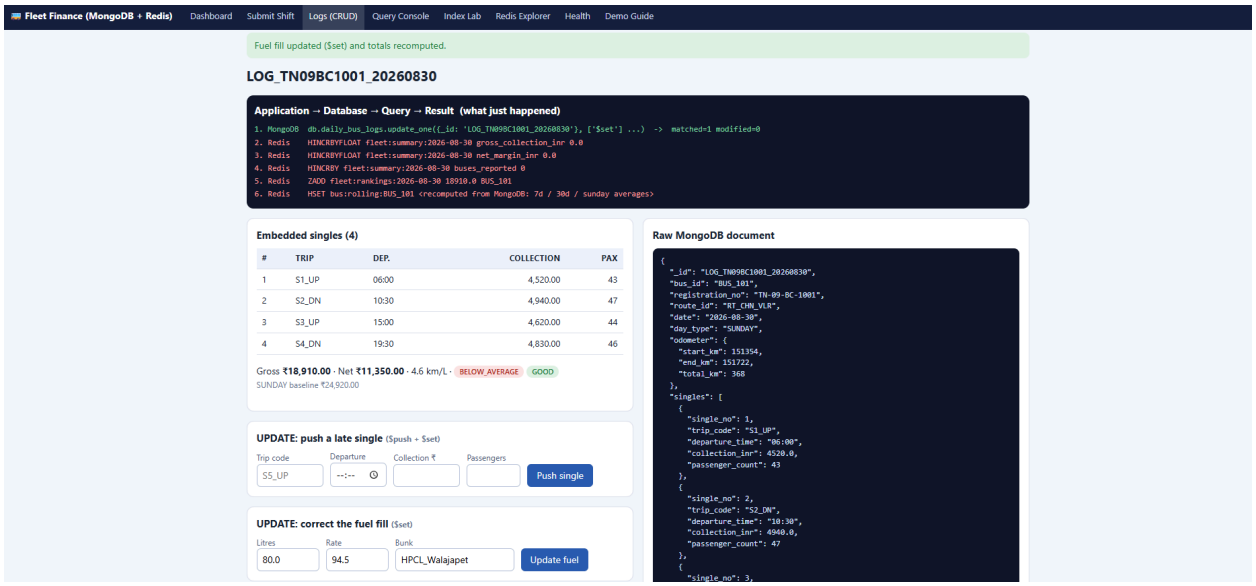
- Flexible schema: the singles array has 3–5 entries per day (600 embedded singles across 150 logs, verify), and per-bus day_type_factors live in the master document. Neither needed a migration.
- Embedding and referencing: both are used deliberately (Section 2). Q11 shows the $lookup join when one is needed.
- Atomic upsert (replace_one(..., upsert=True)) and in-place array update ($push together with $set).
- Redis atomic counters: HINCRBYFLOAT runs inside a MULTI/EXEC transaction.
- Redis sorted set: an auto-ordered leaderboard.
- Hybrid design: MongoDB provides durability and analytics, and Redis provides speed. Redis can always be rebuilt from MongoDB.

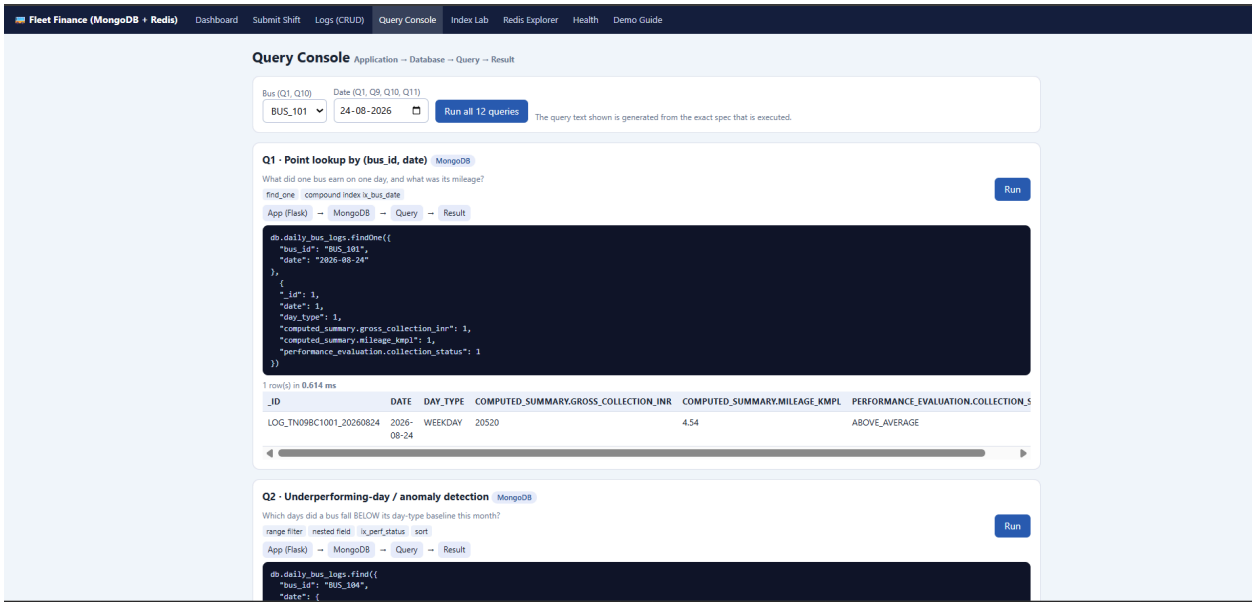
[6] Screenshots

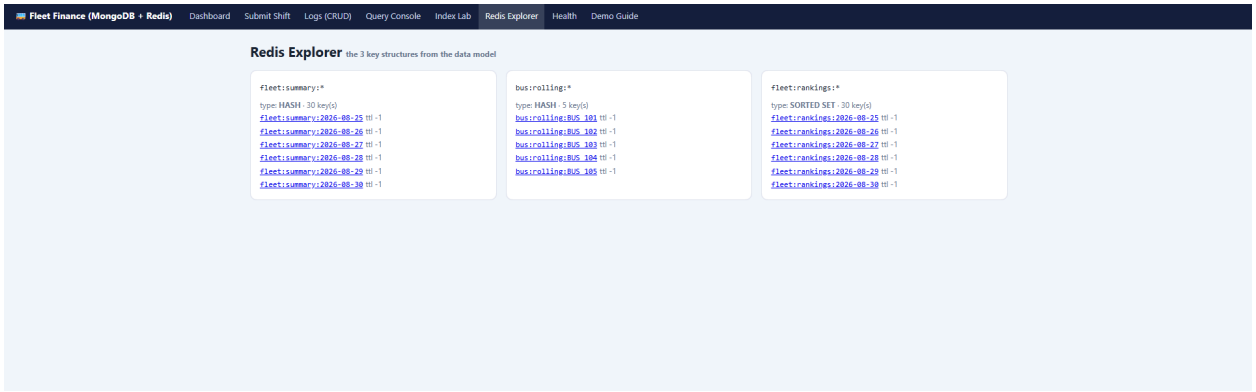


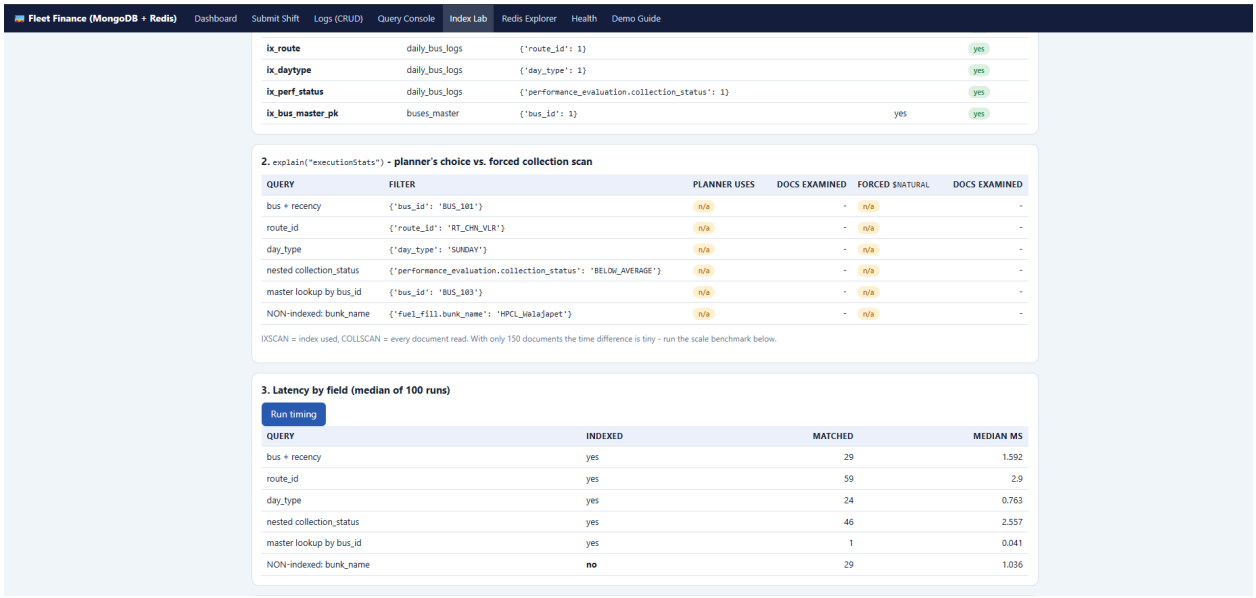



Fleet Finance (MongoDB + Redis)
Dashboard
Submit Shift
Logs (CRUD)
Query Console
Index Lab
Redis Explorer
Health
Demo Guide
Live Dashboard
Date
30-08-2026
Show
+ Submit shift
Application → Database → Query → Result (what just happened)
1. MongoDB db.daily_bus_logs.replace_one({_id: 'LOG_TN09BC1001_20260830'}, <doc>, upsert=True) -> REPLACED existing day (acknowledged=True)
2. Redis HINCRBYFLOAT fleet:summary:2026-08-30 gross_collection_inr 1270.0
3. Redis HINCRBYFLOAT fleet:summary:2026-08-30 net_margin_inr 1411.75
4. Redis HINCRBY fleet:summary:2026-08-30 buses_reported 0
5. Redis ZADD fleet:rankings:2026-08-30 18910.0 BUS_101
6. Redis HSET bus:rolling:BUS_101 <recomputed from MongoDB: 7d / 30d / sunday averages>
Fleet gross · 2026-08-30
₹ 108,210.00
Redis HGETALL · 0.271 ms
Fleet net margin
₹ 65,916.52
Redis hash fleet:summary:2026-08-30
Buses reported
5
4 alert(s) today
Leaderboard
Redis ZREVRANGE · 0.574 ms
BUS GROSS ₹ VS BASELINE COLLECTION MILEAGE
1 BUS_103 23,760.00 -6.8% BELOW_AVERAGE GOOD
2 BUS_105 23,370.00 +7.7% ABOVE_AVERAGE LOW
3 BUS_102 23,340.00 -1.9% AVERAGE GOOD
4 BUS_101 18,910.00 -24.1% BELOW_AVERAGE GOOD
5 BUS_104 18,830.00 -9.1% BELOW_AVERAGE GOOD
Alerts for 2026-08-30 (MongoDB flags)
BUS_101 — collection 18,910.00 below SUNDAY baseline 24,920.00:
BUS_103 — collection 23,760.00 below SUNDAY baseline 25,480.00:
BUS_104 — collection 18,830.00 below SUNDAY baseline 20,720.00:
BUS_105 — mileage 4.48 km/L below 4.8
Rolling averages per bus
Redis HGETALL x5 · 0.932 ms
BUS 7-DAY 30-DAY SUNDAY WEEKDAY MILEAGE 30D UPDATED
BUS_101 18,168.57 18,778.33 24,354.00 17,663.20 4.59 2026-10-10 13:38:50
BUS_102 17,675.71 17,505.67 23,780.00 16,250.80 4.47 2026-10-10 13:32:18
BUS_103 19,805.71 19,268.00 26,208.00 17,880.00 4.75 2026-10-10 13:32:18
BUS_104 15,272.86 15,901.67 20,852.00 14,911.60 4.33 2026-10-10 13:32:18


Fleet Finance (MongoDB + Redis)
Dashboard
Submit Shift
Logs (CRUD)
Query Console
Index Lab
Redis Explorer
Health
Demo Guide
Fuel fill updated ($set) and totals recomputed.
LOG_TN09BC1001_20260830
Application → Database → Query → Result (what just happened)
1. MongoDB db.daily_bus_logs.update_one({_id: 'LOG_TN09BC1001_20260830'}, ['$set'] ...) -> matched=1 modified=0
2. Redis HINCRBYFLOAT fleet:summary:2026-08-30 gross_collection_inr 0.0
3. Redis HINCRBYFLOAT fleet:summary:2026-08-30 net_margin_inr 0.0
4. Redis HINCRBY fleet:summary:2026-08-30 buses_reported 0
5. Redis ZADD fleet:rankings:2026-08-30 18910.0 BUS_101
6. Redis HSET bus:rolling:BUS_101 <recomputed from MongoDB: 7d / 30d / sunday averages>
Embedded singles (4)
TRIP DEP. COLLECTION PAX
1 S1_UP 06:00 4,520.00 43
2 S2_DN 10:30 4,940.00 47
3 S3_UP 15:00 4,620.00 44
4 S4_DN 19:30 4,830.00 46
Gross ₹18,910.00 · Net ₹11,350.00 · 4.6 km/L · BELOW_AVERAGE GOOD
SUNDAY baseline ₹24,920.00
UPDATE: push a late single ($push + $set)
Trip code
S5_UP
Departure
--:--
Collection ₹
Passengers
Push single
UPDATE: correct the fuel fill ($set)
Litres
80.0
Rate
94.5
Bunk
HPCL_Walajapet
Update fuel
Raw MongoDB document
{
"_id": "LOG_TN09BC1001_20260830",
"bus_id": "BUS_101",
"registration_no": "TN-09-BC-1001",
"route_id": "RT_CHN_VLR",
"date": "2026-08-30",
"day_type": "SUNDAY",
"odometer": {
"start_km": 151354,
"end_km": 151722,
"total_km": 368
},
"singles": [
{
"single_no": 1,
"trip_code": "S1_UP",
"departure_time": "06:00",
"collection_inr": 4520.0,
"passenger_count": 43
},
{
"single_no": 2,
"trip_code": "S2_DN",
"departure_time": "10:30",
"collection_inr": 4940.0,
"passenger_count": 47
},
{
"single_no": 3,


Fleet Finance (MongoDB + Redis)
Dashboard
Submit Shift
Logs (CRUD)
Query Console
Index Lab
Redis Explorer
Health
Demo Guide
Query Console
Application → Database → Query → Result
Bus (Q1, Q10)
BUS_101
Date (Q1, Q9, Q10, Q11)
24-08-2026
Run all 12 queries
The query text shown is generated from the exact spec that is executed.
Q1 · Point lookup by (bus_id, date)
MongoDB
What did one bus earn on one day, and what was its mileage?
find_one
compound index ix_bus_date
App (Flask) → MongoDB → Query → Result
Run
db.daily_bus_logs.findOne({
"bus_id": "BUS_101",
"date": "2026-08-24"
},
{
"_id": 1,
"date": 1,
"day_type": 1,
"computed_summary.gross_collection_inr": 1,
"computed_summary.mileage_kmpl": 1,
"performance_evaluation.collection_status": 1
})
1 row(s) in 0.614 ms
_ID
DATE
DAY_TYPE
COMPUTED_SUMMARY.GROSS_COLLECTION_INR
COMPUTED_SUMMARY.MILEAGE_KMPL
PERFORMANCE_EVALUATION.COLLECTION_S
LOG_TN09BC1001_20260824
2026-08-24
WEEKDAY
20520
4.54
ABOVE_AVERAGE
Q2 · Underperforming-day / anomaly detection
MongoDB
Which days did a bus fall BELOW its day-type baseline this month?
range filter
nested field
ix_perf_status
sort
App (Flask) → MongoDB → Query → Result
Run
db.daily_bus_logs.find({
"bus_id": "BUS_104",
"date": {


Fleet Finance (MongoDB + Redis)
Dashboard
Submit Shift
Logs (CRUD)
Query Console
Index Lab
Redis Explorer
Health
Demo Guide
Redis Explorer
the 3 key structures from the data model
fleet:summary:*
type: HASH · 30 key(s)
fleet:summary:2026-08-25 ttl -1
fleet:summary:2026-08-26 ttl -1
fleet:summary:2026-08-27 ttl -1
fleet:summary:2026-08-28 ttl -1
fleet:summary:2026-08-29 ttl -1
fleet:summary:2026-08-30 ttl -1
bus:rolling:*
type: HASH · 5 key(s)
bus:rolling:BUS_101 ttl -1
bus:rolling:BUS_102 ttl -1
bus:rolling:BUS_103 ttl -1
bus:rolling:BUS_104 ttl -1
bus:rolling:BUS_105 ttl -1
fleet:rankings:*
type: SORTED SET · 30 key(s)
fleet:rankings:2026-08-25 ttl -1
fleet:rankings:2026-08-26 ttl -1
fleet:rankings:2026-08-27 ttl -1
fleet:rankings:2026-08-28 ttl -1
fleet:rankings:2026-08-29 ttl -1
fleet:rankings:2026-08-30 ttl -1



3. Latency by field (median of 100 runs)

Run timing

QUERY	INDEXED	MATCHED	MEDIAN MS
bus + recency	yes	29	1.592
route_id	yes	59	2.9
day_type	yes	24	0.763
nested collection_status	yes	46	2.557
master lookup by bus_id	yes	1	0.041
NON-indexed: bunk_name	**no**	29	1.036

4. Scale benchmark - same query without vs. with index

Documents

100000 Run benchmark Creates a throw-away collection, then deletes it.

	PLAN	DOCS EXAMINED	MEDIAN MS
No index	n/a	-	null
With ix_bus_date	n/a	-	null

100,000 docs, query matched 218. Speed-up: **0.9x** (index build 0s)

[7] Results

Findings from the data (verify)

- **Day-type effect (Q4).** Sunday collection runs about **1.4–1.5× weekday** collection on every bus. For BUS_101 it is ₹25,634 against ₹17,663, and for BUS_102 ₹23,780 against ₹16,251. Judging Sundays against their own baseline avoids false alerts that a flat average would raise.
- **Collection status.** Of the 150 bus-days, 44 were ABOVE_AVERAGE, 60 AVERAGE and 46 BELOW_AVERAGE (about 31% flagged for review).
- **Fuel efficiency.** 27 of 150 bus-days (18%) had LOW mileage, and 80 were GOOD.
- **Route profitability (Q7).** RT_CHN_VLR is the most profitable route.
- **Cost per km (Q12).** BUS_103 earns the most per km at ₹54.97, with fuel at ₹19.87 per km, while BUS_101 earns ₹51.90 and spends ₹20.65 per km.
- **Peak slots (Q5).** Early up-trips (S1_UP) are the highest-earning slots.

System results

- **Correctness:** `run_all.py` reports every check PASS: document counts, 5 indexes present, Redis consistent with MongoDB after seeding and after CRUD, and all 12 queries run. 12 automated tests pass.
- **Idempotence:** re-submitting the same bus and date leaves exactly one document and the right Redis totals, with no double counting.
- **Resilience:** with Redis stopped, the dashboard still works from MongoDB, and “Rebuild Redis” restores the cache.

[8] Limitations and Scope

Limitations

- Single-node deployment: no replica set (no automatic failover) and no sharding.
- No authentication or role-based access.
- Money is stored as floating-point; Redis `HINCRBYFLOAT` can accumulate tiny rounding differences, so values are rounded for display and the rebuild resets them.
- Dates are ISO strings rather than BSON `Date`.
- Rolling averages use the last N logs, not calendar windows.
- Redis is eventually consistent with MongoDB: there is no distributed transaction, and the rebuild and audit mitigate this.
- Data is seeded and simulated, not from a live operator.

Future scope

- Replica set with change streams feeding Redis automatically.
- MongoDB time-series collections and BSON dates.
- `Decimal128` or integer paise for money.
- Driver-wise and conductor-wise analytics, alerts by SMS or WhatsApp, and a mobile entry app.
- Authentication and role-based access.
- A predictive layer (expected collection by route, day and season).

[9] Individual contribution of each team member

Abhinavanagarajan A (23BCE1123): Redis implementation and exercise	`redis_layer.py`. Designed and implemented the three Redis structures, the atomic `HINCRBYFLOAT` live totals, the `ZADD/ZREVRANGE` leaderboard, delta-based updates, the rolling-average refresh, and the rebuild and consistency-audit routines. Also built the Redis Explorer data layer. Alongside `service.py`, `db.py`,`logic.py`.
Rakshana V (23BCE1620): MongoDB implementation	`generate_data.py`, `queries_demo.py`, `indexing_demo.py`. Designed the document model (embedding vs. referencing), generated the deterministic dataset, wrote the 12 queries and 5 aggregation pipelines, and defined the 5 indexes with their `explain()` and benchmark evidence. Also wrote the baseline and day-type evaluation rules.
Vishnu Swaroop G (23BCE1129): Integration	`app.py`,`crud_demo.py`, `run_all.py`, templates and tests. Built the Flask application, the single MongoDB-then-Redis write path, the CRUD operations with optimistic concurrency, the graceful-degradation and error handling, the Query Console and Health pages, and the reproducible end-to-end run with PASS/FAIL checks and the test suite.

GitHub Source Repository

https://github.com/abhinavanagarajan/fleet-tracker

Live Demonstration [Mandatory]	Completed on 21-10-2026
Individual Viva [Mandatory]	Completed on 21-10-2026